Glossary of Technical Terms

Companion to "In-House Legal Intelligence System" (capstone_phase1.pdf)

Manish Kumar (251330604)

CS-781, IIT Kanpur

Every term from the main deck, with an intuitive example

How to read this glossary

- Each term has a short **definition** and a green **Example**.
- Examples use small numbers you can check by hand, or everyday analogies.
- Most examples come from legal text, because that is what our model reads.
- Sections follow the order of the main deck. An alphabetical index is on the last slides.

A running example

Many examples use this sentence from a judgment:

"The appellant filed an appeal against the order of the High Court under Section 302."

Contents

1. The project and the competition
2. The LoRA family in depth
3. Evaluation metrics
4. Data engineering
5. Tokenization
6. Language modelling basics
7. Transformer building blocks
8. Architectures
9. Training
10. Decoding
11. Knowledge distillation
12. Experiments and code engineering
13. Index

Foundation model and self-hosting

Foundation model

A general model trained once on a large corpus, then *adapted* to many tasks. It learns vocabulary, grammar and style, but no single task.

Example. Our Phase-1 legal LM is the foundation. Phase 2 adds an entity-tagging head on top of it, Phase 3 a sentence classifier, and so on. It is like a law graduate who later specialises in tax or criminal law.

Self-hosted model

A model that runs on hardware the organisation controls. No input text is sent to an outside company.

Example. A partner pastes a confidential client contract into the firm's own server. With a cloud API, that contract would travel to someone else's data centre.

Codabench, leaderboards and submissions

Codabench and the leaderboard

An online platform that hosts competitions. You upload predictions, it scores them against hidden answers, and ranks everyone on a public leaderboard.

Example. Like an exam where the answer key stays with the examiner. You hand in your answer sheet and get back only your mark and rank.

prediction.json, submission.zip and test IDs

The answer file maps each official test ID to a prediction. It must sit at the *root* of a zip archive. Every ID must appear, and no unknown ID may appear.

Example. If the test has IDs 1, 2 and 3, then a file with only 1 and 2 fails. So does a file with 1, 2, 3 and 4. A zip holding `results/prediction.json` also fails.

Datasets and access

Hugging Face Hub and gated dataset

The Hub is a website that hosts models and datasets. A *gated* dataset needs you to accept its terms and log in with a personal access token first.

Example. Like a library's rare-books room. You sign the register once, show your card, and then you may read.

Train, validation, dev and test splits

Train is used to learn. **Validation** tracks progress and picks checkpoints. **Dev** is our local copy of the task format. **Test** is scored only by the organisers.

Example. A student practises on past papers (train), takes weekly mock tests (validation and dev), and sits the final exam once (test).

Adaptation vocabulary

Pre-training vs fine-tuning

Pre-training teaches general language from raw text. **Fine-tuning** continues training on labelled data for one task.

Example. Pre-training: read 200M tokens of judgments. Fine-tuning: then learn from 5,000 sentences labelled “Fact” or “Issue”.

PEFT (parameter-efficient fine-tuning)

Freeze most of the model and train only a small number of new or selected parameters.

Example. An 82M-parameter model where only 0.3M parameters change. You store one shared base plus a tiny file per task, instead of seven full copies.

PEFT methods (1): the LoRA family at a glance

Method	One-line summary
LoRA	learn a low-rank update $W_0 + \frac{\alpha}{r} BA$; only A and B are trained
QLoRA	LoRA on top of a base model stored in 4-bit NF4
AdaLoRA	SVD-shaped LoRA that prunes rank from unimportant matrices
rsLoRA	LoRA scaled by $\alpha/\sqrt{r}$, so large ranks keep learning
LORMA	multiplicative low-rank update $(I + BA)\, W_0$; compulsory

The full treatment, with definitions, mathematics, worked examples, advantages, disadvantages and applications, is in the section **"The LoRA family in depth"**.

PEFT methods (2): prompts, adapters and sparse updates

Method	What is trained	Intuitive example
Prompt tuning	a few learned "soft token" vectors placed before the input	20 invisible words that tell the model "this is an NER task"
Prefix tuning	learned key/value vectors added to *every* layer's attention	the same hint, whispered at every floor of the building
Houlsby adapter	two small bottleneck MLPs per layer (after attention and after the MLP)	plug-in modules: $768 \to 64 \to 768$ each
Pfeiffer adapter	one bottleneck MLP per layer, after the MLP	half as many plug-ins as Houlsby
BitFit	only the bias terms	A4 has *no* biases, so BitFit would change nothing there
IA^3	three vectors that rescale keys, values and MLP activations	volume knobs on each channel, not new wiring
Layer freezing	only the top k layers	keep the "grammar" layers fixed, retrain the "task" layers
Full fine-tuning	every parameter	the most flexible, and the most memory-hungry

The later-phase tasks (1)

NER and strict span matching (Phase 2)

Named-entity recognition finds spans of text and gives each a type. Spans are written as character offsets `[start, end)`. "Strict" means both offsets and the label must match exactly.

Example. In "Appeal by Ram Kumar", gold is `[10,19,"APP"]`. Predicting `[10,18,"APP"]` ("Ram Kuma") scores zero, even though it is nearly right.

Rhetorical roles (Phase 3)

The function a sentence plays in a judgment: a fact, an issue, an argument, a ruling, and so on. There is one label per sentence.

Example. "The accused was arrested on 3 May" is a *Fact*. "Whether Section 302 applies" is an *Issue*. "The appeal is dismissed" is a *RulingByPresentCourt*.

The later-phase tasks (2)

Judgment prediction and multi-label (Phases 4–5)

CJPE predicts one binary outcome (0 or 1) per case. LSI is *multi-label*: a case can have several correct statute IDs at once.

Example. CJPE: appeal accepted = 1, rejected = 0. LSI: a murder case with a conspiracy charge may need both statute 302 and statute 120B.

Threshold and calibration

A multi-label model outputs a probability per label. The threshold decides which labels to keep. Calibration makes a 0.7 score really mean "right about 70% of the time".

Example. Scores are 0.81, 0.46 and 0.12. A threshold of 0.5 keeps one statute, while 0.4 keeps two. A per-label threshold lets rare statutes use a lower bar.

The later-phase tasks (3)

Abstractive vs extractive summarisation (Phase 6)

Extractive copies important sentences. **Abstractive** writes new sentences in its own words.

Example. Extractive: "The appeal is dismissed with costs." Abstractive: "The court rejected the appeal and ordered the appellant to pay costs."

Prior case retrieval and hard negatives (Phase 7)

Rank candidate past cases by relevance to a query case. A *hard negative* is an irrelevant case that looks relevant. Contrastive learning pulls relevant pairs together and pushes others apart.

Example. Query: a bail case. A hard negative is another bail case on a different legal question. It shares many words but is not a precedent for this one.

Long-document strategies

Chunking and sliding windows

Split a long document into pieces that fit the context window. *Overlapping* windows share some tokens, so no sentence is cut without context.

Example. A 5,000-token judgment with a 1,024-token window and a 256-token overlap gives chunks $[0, 1024)$, $[768, 1792)$, $[1536, 2560)$, and so on.

Pooling and hierarchical models

Pooling merges many vectors into one, by mean, max, or learned attention weights. A **hierarchical** model encodes chunks first, then combines the chunk vectors.

Example. Five chunk vectors are averaged into one document vector for CJPE. Attention pooling might instead weight the "reasoning" chunk at 0.6 and the "facts" chunk at 0.1.

The LoRA family: roadmap

Phases 2–7 need at least five fine-tuning techniques each, and LORMA is compulsory. Most of the menu belongs to one family, built on a single idea: *learn a small, low-rank change to each weight matrix*.

Method	What it changes about LoRA
LoRA	the base idea: $W + BA$
QLoRA	stores the frozen W in 4 bits
AdaLoRA	moves rank to where it helps
rsLoRA	fixes the scaling factor
LORMA	multiplies instead of adds

For each method, this section gives

1. a definition and intuition;
2. the maths, with a worked example;
3. advantages;
4. disadvantages;
5. applications, in general and in this capstone.

Running model

Examples use our A1 model: $d = 768$, 8 layers, 82.1M parameters.

Why not just fine-tune everything?

Memory for full fine-tuning with AdamW (fp32), per parameter:

$$\underbrace{4}_{\text{weight}} + \underbrace{4}_{\text{gradient}} + \underbrace{8}_{\text{Adam } m,\, v} = 16 \text{ bytes}$$

For A1: $82.1\text{M} \times 16 \approx 1.3$ GB, before activations.

Storage: one full checkpoint per experiment is about 330 MB. Phases 2–7 need at least 36 experiments, so about 12 GB of near-identical copies.

Forgetting: updating every weight on 5,000 NER documents can erase general legal knowledge learned from 200M tokens.

The key observation (Aghajanyan et al., 2021; Hu et al., 2021)

The *change* a task needs, $\Delta W = W_{\text{tuned}} - W_{\text{pre}}$, has a low **intrinsic rank**. A few directions of change are enough.

Intuition

A general legal writer becomes an NER tagger by learning a few new habits, such as "notice party names" and "notice section numbers". It does not need to relearn English. Those few habits are the few "directions" of ΔW.

Background: rank and low-rank matrices

Rank is the number of independent directions a matrix can produce. A $d \times d$ matrix of rank r can be written as a product

$$\underbrace{\Delta W}_{d\times d} = \underbrace{B}_{d\times r}\ \underbrace{A}_{r\times d},$$

which needs $2dr$ numbers instead of d^2.

SVD (singular value decomposition): any matrix is $U\Sigma V^\top$. The singular values in Σ say how important each direction is. Keeping the largest r gives the best rank-r approximation.

Rank-1 example ($d = 3$)

$B = \begin{bmatrix} 0.5 \\ 0 \\ -0.5 \end{bmatrix}$, $A = \begin{bmatrix} 1 & 0 & 1 \end{bmatrix}$ gives

$$BA = \begin{bmatrix} 0.5 & 0 & 0.5 \\ 0 & 0 & 0 \\ -0.5 & 0 & -0.5 \end{bmatrix}$$

Six numbers define nine entries. Every row is a multiple of $[1, 0, 1]$: **one** direction.

At $d = 768$ and $r = 8$: $2 \times 768 \times 8 = 12{,}288$ numbers instead of 589,824.

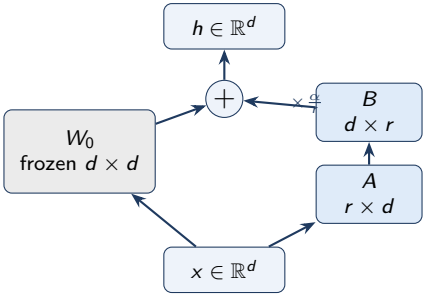
LoRA: definition and intuition

Definition (Hu et al., 2021)

Low-Rank Adaptation freezes every pretrained weight W_0. Next to chosen linear layers it adds a trainable low-rank path BA. Only A and B are trained. The output is the frozen output plus a small learned correction.

Analogy

The pretrained model is a printed statute book. LoRA does not reprint the book. It adds thin *amendment slips* beside the sections that need changing. Each slip is cheap, and removing it restores the original.



Frozen path in grey, trainable path in blue.
$r \ll d$, e.g. $r = 8$ and $d = 768$.

LoRA: the mathematics

Forward pass

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} B\,(A\,x)$$

- $A \in \mathbb{R}^{r \times d_{\text{in}}}$ compresses x to r numbers.
- $B \in \mathbb{R}^{d_{\text{out}} \times r}$ expands them back.
- α / r is a fixed **scaling factor**; α is a hyperparameter.

Initialisation: $A \sim \mathcal{N}(0, \sigma^2)$ (or Kaiming), and $B = 0$. So $\Delta W = 0$ at step 0, and the model starts *exactly* as the pretrained one.

Gradients: W_0 gets none. Only $\frac{\partial \mathcal{L}}{\partial A}$ and $\frac{\partial \mathcal{L}}{\partial B}$ are computed and stored by the optimizer.

Worked forward pass ($d = 3$, $r = 1$, $\alpha = 1$)

$W_0 = I$, $x = [2, 1, 0]^\top$, with A and B from the rank-1 slide.

$$Ax = 1 \cdot 2 + 0 \cdot 1 + 1 \cdot 0 = 2$$

$$B(Ax) = [1,\ 0,\ -1]^\top$$

$$h = [2, 1, 0]^\top + [1, 0, -1]^\top = [3, 1, -1]^\top$$

The correction costs $2dr = 6$ multiplications, against $d^2 = 9$ for the frozen path.

LoRA: counting parameters and memory for A1

LoRA on the query and value projections (`attn.q`, `attn.v`) of all 8 layers, with $r = 8$:

$$8 \text{ layers} \times 2 \text{ matrices} \times (768 \times 8 + 8 \times 768) = 196{,}608$$

That is **0.24%** of 82.1M.

	Full FT	LoRA $r = 8$
Trainable params	82.1M	0.20M
Adam state + grads	985 MB	2.4 MB
Saved per task	328 MB	0.8 MB

fp32; the frozen weights (328 MB) are shared by all tasks.
Activation memory is still needed for the backward pass.

Trainable fraction by target

With $r = 8$ in A1, per layer:

- q, v: 24,576
- q, k, v, o: 49,152
- all linear layers (adds `fc` $768 \to 3072$ and `proj` $3072 \to 768$): 110,592

Over 8 layers that is 0.2M, 0.4M or 0.9M. All are tiny next to 82M.

Note

The new task head, e.g. a 768×13 classifier for Phase 3, is always fully trained.

LoRA: hyperparameters and where to apply it

Knob	Meaning	Typical choice and effect
Rank r	number of directions of change	4–64. Higher: more capacity, more overfitting risk
Alpha α	strength of the update, via α/r	often $\alpha = 2r$ or 16. With fixed α, changing r changes the step size
Targets	which linear layers get BA	q, v is the original recipe. All linear layers usually work better at little cost
LoRA dropout	dropout on x before A	0.05–0.1 for small datasets such as NER folds
Learning rate	for A and B only	about $10\times$ the full fine-tuning rate, e.g. 2×10^{-4} to 10^{-3}
Trainable extras	modules left unfrozen	the task head always, sometimes LayerNorms

Mapping to our code

In our code LoRA would wrap `Attention.q/k/v/o`, `FeedForward.fc/proj` and `SwiGLU.gate/up/proj`. Embeddings and norms stay frozen.

LoRA: merging, task switching and inference

Merging. After training, fold the update into the weight once:

$$W' = W_0 + \tfrac{\alpha}{r} BA$$

The merged model has the *original* shape and speed. There is **zero extra inference latency**.

Un-merging. $W_0 = W' - \tfrac{\alpha}{r} BA$ restores the base model.

Task switching. Keep one base model and one 0.8 MB adapter per task. Swap adapters in milliseconds.

Capstone example

One A4 base model in GPU memory serves:

- the NER adapter (Phase 2);
- the rhetorical-role adapter (Phase 3);
- the judgment adapter (Phase 4); and so on.

The firm deploys *one* model with small add-ons, not seven 330 MB models.

Careful

Measure latency *after* merging. An unmerged adapter adds two small matrix products per wrapped layer.

LoRA: advantages, disadvantages and applications

Advantages

- 100–10,000$\times$ fewer trainable parameters.
- Much less optimizer memory, so larger batches fit on a T4.
- No inference latency after merging.
- Tiny per-task files; easy to swap.
- Starts exactly at the pretrained model ($B = 0$), so training is stable.
- Less forgetting of pretrained knowledge.
- Often matches full fine-tuning.

Disadvantages

- Low rank limits capacity. Tasks far from pretraining may need full fine-tuning.
- Extra hyperparameters: r, α, targets.
- The same rank everywhere wastes budget (fixed by AdaLoRA).
- α/r scaling makes high ranks learn slowly (fixed by rsLoRA).
- Activation memory is unchanged; long sequences still cost.
- Merged adapters for several tasks cannot be batched together.

Applications

- Instruction and domain tuning of large LLMs.
- Image-model styles, e.g. Stable Diffusion LoRAs.
- Serving many customers from one base model.
- **Capstone**: listed for Phases 2, 3 and 6, and the baseline every other family member is compared with. Examples: LoRA on q, v plus a token head for NER; LoRA on all layers plus a sentence head for RR.

QLoRA: definition and intuition

Definition (Dettmers et al., 2023)

Quantised LoRA: store the frozen base weights in **4-bit NormalFloat (NF4)**, and train ordinary LoRA adapters in 16-bit on top. Gradients flow *through* the 4-bit weights into A and B, but the 4-bit weights never change.

Three ingredients:

1. **NF4**: a 4-bit number format suited to normally distributed weights.
2. **Double quantisation**: the scaling constants are quantised too.
3. **Paged optimizers**: optimizer state spills to CPU RAM instead of crashing on memory spikes.

Analogy

The statute book is photocopied at low resolution to fit in your bag. You can still read it clearly enough to write the amendment slips, and the slips themselves are written in full detail.

Headline result

A 65B model was fine-tuned on one 48 GB GPU, matching 16-bit quality on the paper's benchmarks.

QLoRA: NF4 quantisation

Block-wise absmax quantisation. Split W into blocks of 64 weights. For each block:

$$c = \max_i |w_i|, \qquad q_i = \arg\min_{v \in \mathrm{NF4}} \left| \frac{w_i}{c} - v \right|$$

Store the 4-bit index of q_i and one constant c per block. **Dequantise** when needed: $\hat{w}_i = c \cdot q_i$.

NF4 levels: 16 values in $[-1, 1]$ at quantiles of $\mathcal{N}(0, 1)$, including an exact 0. They are dense near 0, where most weights lie:

$-1,\ -.70,\ -.53,\ -.39,\ -.28,\ -.18,\ -.09,\ 0,$
$.08,\ .16,\ .25,\ .34,\ .44,\ .56,\ .72,\ 1$

Worked example (block of 4)

$w = [0.12, -0.40, 0.05, 0.33]$, so $c = 0.40$.

$$w/c = [0.30, -1.00, 0.125, 0.825]$$

Nearest NF4 levels: $[0.34, -1, 0.16, 0.72]$.

$$\hat{w} = [0.135, -0.40, 0.064, 0.289]$$

The errors are at most 0.04, and each weight now needs 4 bits instead of 16.

QLoRA: forward pass, double quantisation and paging

Forward pass (compute in bf16):

$$h = \text{dequant}(W_0^{\text{NF4}})\, x + \tfrac{\alpha}{r} BAx$$

Backward pass: gradients flow through the dequantised W_0 to reach A and B. W_0 itself is never updated.

Paged optimizers: Adam states live in memory that can page to CPU RAM. A long document that briefly spikes memory slows down instead of crashing.

Double quantisation

One fp32 constant per 64 weights adds $32/64 = 0.5$ bits per weight. Quantising the constants to 8 bits, in blocks of 256, cuts this to about 0.127 bits.

Memory for A1's frozen weights

fp32	328 MB
fp16 / bf16	164 MB
NF4 (+ constants)	$\approx$ 42 MB

QLoRA: advantages, disadvantages and applications

Advantages

- About 4× less memory for frozen weights than fp16, and 8× less than fp32.
- Fine-tunes models that would not otherwise fit on one GPU.
- Quality close to 16-bit LoRA in the paper.
- Same small adapter files as LoRA.
- Paged optimizers survive memory spikes from long documents.

Disadvantages

- Slower per step: weights are dequantised on every forward and backward pass.
- Needs a CUDA GPU and the bitsandbytes kernels, not CPU or MPS.
- Quantisation error slightly perturbs the base model.
- Merging into 4-bit weights loses precision. You usually merge into a dequantised 16-bit copy.
- **Small gain for small models.** A1 in NF4 is about 41 MB instead of 164 MB (fp16), while activations dominate memory.

Applications

- Fine-tuning 7B–70B LLMs on consumer or single data-centre GPUs.
- Low-budget academic research.
- **Capstone**: listed for Phases 3, 5, 6 and 7. Useful for your largest Phase-1 model (up to 300M), or a teacher model in distillation.
- Analyse the speed–memory–quality trade-off against 16-bit LoRA, since the memory saving at 80M is modest.

AdaLoRA: definition and intuition

Definition (Zhang et al., 2023)

Adaptive budget allocation for LoRA. Instead of the same rank in every matrix, start with a generous rank everywhere. During training, measure how important each rank component is, and *prune* the least important ones until a total budget is reached.

Analogy

A chambers has 16 junior advocates to assign across 8 matters. At first everyone gets two. After a month, the complex matters get four and the routine ones get none.

Why it helps

Not all matrices matter equally. The paper found that the MLP layers and the *top* layers usually need more rank than the bottom attention layers. A uniform rank wastes parameters where they are not needed and starves the layers that need them.

Budget example

4 matrices start at rank 8 each (32 components). The target budget is 16. After pruning: top-layer W_v has 7, top-layer MLP 5, bottom W_v 3, bottom W_q 1.

AdaLoRA: the mathematics (1): SVD form and regulariser

SVD-shaped update

$$\Delta W = P \Lambda Q$$

with $P \in \mathbb{R}^{d \times r}$, $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_r)$ and $Q \in \mathbb{R}^{r \times d}$. Component i is the triplet $(P_{:,i}, \lambda_i, Q_{i,:})$.

Removing component i means setting $\lambda_i = 0$. P and Q stay intact, so the component can come back later.

Keep P and Q near-orthogonal:

$$R(P, Q) = \|P^\top P - I\|_F^2 + \|QQ^\top - I\|_F^2$$

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \gamma \sum_{\text{matrices}} R(P, Q)$$

Why SVD form, not BA?

In BA, the rank directions are mixed together, so you cannot tell which one to delete. In $P\Lambda Q$ with orthogonal P and Q, each λ_i is the strength of one clean direction. Zeroing λ_i removes exactly one direction, like dropping one singular value in an SVD.

AdaLoRA: the mathematics (2): importance and budget

Importance of a weight w (sensitivity, smoothed over steps):

$$s(w) = \left| w \cdot \frac{\partial \mathcal{L}}{\partial w} \right|, \quad \bar{s}_t = \beta_1 \bar{s}_{t-1} + (1 - \beta_1) s_t$$

This estimates how much the loss would change if w were removed. An uncertainty term discounts estimates that jump around.

Score of component i: the importance of λ_i, plus the average importance of P's column i and Q's row i.

Budget schedule: the total rank $b(t)$ starts at about $1.5\times$ the target, is held during warm-up, then shrinks along a cubic curve to the target. At each step the lowest-scoring λ_i are zeroed.

Sensitivity example

Component 1: $\lambda_1 = 0.9$, gradient 0.5, so $s = 0.45$.

Component 2: $\lambda_2 = 0.8$, gradient 0.01, so $s = 0.008$.

Changing component 2 barely changes the loss. It is pruned first, even though its λ is large.

AdaLoRA: advantages, disadvantages and applications

Advantages

- Better accuracy than LoRA at the same parameter budget, especially at small budgets.
- Removes the need to hand-tune rank per layer.
- The final rank pattern is *interpretable*: it shows which layers the task needed.
- Pruned components can be revived during training.
- Merges into W like LoRA, with no inference cost.

Disadvantages

- More moving parts: budget schedule, warm-up, regulariser weight γ, smoothing β.
- Extra computation for importance scores and the orthogonality penalty; slower than LoRA.
- Importance estimates are noisy on small datasets.
- Needs a known number of training steps for the schedule.
- Harder to implement correctly by hand.

Applications

- Tight parameter budgets, e.g. many tasks served on-device.
- Studying which layers a task relies on.
- **Capstone**: listed for Phases 3, 4, 5 and 6.
- Report the final rank map, e.g. "CJPE put most rank in the top four MLP layers". This is ready-made analysis for the report.

rsLoRA: definition and the scaling problem

Definition (Kalajdzievski, 2023)

Rank-stabilised LoRA is LoRA with one change: the scaling factor.

$$\text{LoRA: } \gamma_r = \frac{\alpha}{r} \qquad \text{rsLoRA: } \gamma_r = \frac{\alpha}{\sqrt{r}}$$

Why. $BAx = \sum_{i=1}^{r} b_i \, (a_i^\top x)$ is a sum of r roughly independent terms, so it grows like $\sqrt{r}$.

- $\times \frac{1}{r}$: the update shrinks like $\frac{1}{\sqrt{r}}$, so a large rank learns *more slowly*.
- $\times \frac{1}{\sqrt{r}}$: the update stays the same size for any r.

The paper proves this keeps learning stable as r grows.

Numbers ($\alpha = 16$)

r	α/r	$\alpha/\sqrt{r}$
4	4.00	8.00
16	1.00	4.00
64	0.25	2.00
256	0.0625	1.00

With plain LoRA at $r = 256$, the update is scaled down $64\times$ compared with $r = 4$. More rank then gives almost no benefit, which misleads rank experiments.

rsLoRA: advantages, disadvantages and applications

Advantages

- A one-line change; no new parameters or cost.
- Higher ranks actually improve results, so a compute–quality trade-off becomes available.
- Learning rate transfers across ranks more reliably.
- Merges exactly like LoRA.

Disadvantages

- No benefit at small ranks ($r \leq 8$), where it behaves like LoRA with a larger α.
- High ranks cost more memory and time.
- Larger effective step sizes can destabilise training if the learning rate is not lowered.
- On its own it may look like a hyperparameter change. Argue clearly why it is a different method.

Applications

- Whenever a task needs a large rank: complex domain shift, generation tasks.
- Fair rank-sweep studies.
- **Capstone**: listed for Phase 3. Natural for Phase 6 summarisation, which may need more capacity than classification.
- A clean experiment: LoRA and rsLoRA at $r \in \{8, 32, 128\}$, all else fixed. Does F1 keep rising with rank?

LORMA: definition and intuition

Definition (Bihany, Patel and Modi, 2025)

Low-Rank Multiplicative Adaptation. Instead of *adding* a low-rank matrix to the frozen weight, LORMA *multiplies* the frozen weight by a transformation built from low-rank factors:

LoRA: $W' = W_0 + BA$ LORMA: $W' = \mathcal{T}(BA)\, W_0$

Geometric view

A matrix multiplication can rotate and rescale. $\mathcal{T}(BA)\, W_0$ applies a learned rotation and scaling to the *outputs* of W_0, reusing all the structure W_0 already has. An additive update instead has to express the change from scratch.

Analogy

LoRA pastes an amendment slip *next to* a section. LORMA applies a *re-interpretation rule to* the section: "read every mention of 'court' as 'tribunal'". The original text is transformed, not supplemented.

Compulsory in Phases 2–7

The course lists LORMA as compulsory in every adaptation phase. It comes from the course team's own paper, so expect viva questions on it.

LORMA: the mathematics (1): the rank problem

The problem. A naive multiplicative update $W' = (BA)\, W_0$ has

$$\text{rank}(BA\, W_0) \leq r,$$

so a full-rank W_0 would collapse to rank r, and the model would lose almost everything it knew.

Rank inflation fixes this. Two strategies are proposed:

- **Additive identity**: $W' = (I + BA)\, W_0$. With $B = 0$ at the start, $W' = W_0$ exactly. The product stays full rank, and the low-rank part only *adjusts* it.
- **Permutation-based**: rearrange the entries of BA so the transform has a higher rank than r, without extra parameters.

Rank collapse, concretely

$d = 768$ and $r = 8$. W_0 has rank 768: it can produce outputs in 768 independent directions. $(BA)W_0$ can produce only 8. Every attention head would see nearly the same few directions.

With $(I + BA)W_0$ the identity keeps all 768 directions, and BA tilts 8 of them.

Check the paper

This summarises the idea. Read the paper for the exact parameterisation, initialisation and scaling before implementing it.

LORMA: the mathematics (2): an efficient forward pass

Never form a $d \times d$ matrix. Compute the frozen output first, then correct it:

$$h = (I + BA)W_0 x = u + B(Au), \qquad u = W_0 x$$

Extra cost: $2dr$ multiplications per token, the same as LoRA.

Difference from LoRA: LoRA's correction reads the *input* x. LORMA's reads the *output* $u = W_0 x$, the already-processed signal.

Merging: compute $W' = (I + BA)W_0$ once after training, so inference costs nothing extra.

Worked example ($d = 3$, $r = 1$)

$u = W_0 x = [2, 1, 0]^\top$, $A = [1, 0, 1]$, $B = [0.5, 0, -0.5]^\top$.

$$Au = 2, \quad B(Au) = [1, 0, -1]^\top$$

$$h = u + B(Au) = [3, 1, -1]^\top$$

This equals the LoRA example only because $W_0 = I$ there. For any other W_0 the two methods give different outputs.

LORMA: advantages, disadvantages and applications

Advantages

- Same number of trainable parameters as LoRA at the same r.
- Reuses the structure of W_0 by transforming it, a different inductive bias.
- The paper reports competitive or better results than LoRA, and faster convergence in some settings.
- Mergeable: $W' = (I + BA)W_0$ can be computed once, so there is no inference cost.
- Starts exactly at the pretrained model when $B = 0$.

Disadvantages

- Needs rank inflation. Without it, the model collapses.
- Unmerged training costs an extra pass through W_0's outputs, $B(A(W_0x))$.
- Gradients for A and B depend on W_0, so scaling and learning rate may need retuning relative to LoRA.
- Less tooling: mainstream PEFT libraries do not provide it, so you must implement it yourself.
- Fewer independent results than LoRA.

Applications

- Parameter-efficient adaptation wherever LoRA is used.
- **Capstone**: compulsory in Phases 2–7, and on all four Phase-1 architectures.
- Best report experiment: LoRA vs LORMA at identical r, targets and budget, on each architecture.
- Compare convergence speed (loss vs step), not only final F1.

The LoRA family side by side

Method	Update	Trainable params	Main benefit	Main cost
LoRA	$W_0 + \frac{\alpha}{r} BA$	$2dr$ per matrix	simple, mergeable, strong default	fixed rank; weak scaling at high r
QLoRA	$\text{deq}(W_0^{\text{NF4}}) + \frac{\alpha}{r} BA$	same as LoRA	4-bit frozen weights; big models fit	slower; CUDA only; small gain at 80M
AdaLoRA	$W_0 + P\Lambda Q$, pruned	starts $\approx 1.5\times$ budget, ends at budget	rank goes where needed	complex schedule; slower
rsLoRA	$W_0 + \frac{\alpha}{\sqrt{r}} BA$	same as LoRA	high ranks keep learning	needs a larger r to matter
LORMA	$(I + BA)\, W_0$	same as LoRA	multiplicative; reuses W_0	rank inflation; self-implemented

Choosing quickly

Start with LoRA. Use rsLoRA if raising r stops helping. Use AdaLoRA when parameters are scarce or you want a layer-importance map. Use QLoRA when memory, not quality, is the limit. Use LORMA always, since it is required, and compare it against LoRA at matched settings.

Using the LoRA family in Phases 2–7

A recipe that satisfies the rules

1. Load each Phase-1 model (A1–A4) and freeze it.
2. Add the task head (token, sentence or document).
3. Wrap the chosen linear layers with LoRA, LORMA, AdaLoRA, and so on.
4. Train the head and adapters only.
5. Log trainable parameters, memory, time and the task metric.
6. Merge, then measure inference speed.

Each method is a **different technique**. Changing only r or α is a hyperparameter change and does not count.

A fair comparison table for the report

Method	Trainable	Memory	Macro-F1
Full FT	82.1M	...	...
LoRA $r = 8$	0.20M	...	...
rsLoRA $r = 64$	1.57M	...	...
AdaLoRA $b = 8$	0.20M	...	...
LORMA $r = 8$	0.20M	...	...

Fill in from your runs, with identical data, head and epochs.

Pitfalls

Forgetting $B = 0$; too low a learning rate; ignoring the α/r effect when changing r.

Precision, recall and F1

$$P = \frac{\text{correct predictions}}{\text{all predictions}}$$

$$R = \frac{\text{correct predictions}}{\text{all gold items}}$$

$$F_1 = \frac{2PR}{P+R} \quad \text{(harmonic mean)}$$

Precision asks: of what I said, how much was right? **Recall** asks: of what was there, how much did I find?

Example (NER)

The document has 14 gold entities. The model predicts 10, and 7 of them are exactly right.

$$P = \tfrac{7}{10} = 0.70, \quad R = \tfrac{7}{14} = 0.50$$

$$F_1 = \frac{2(0.70)(0.50)}{1.20} = 0.583$$

The harmonic mean punishes imbalance. $P = 1$ with $R = 0.01$ gives $F_1 \approx 0.02$, not 0.5.

Macro, micro and subset accuracy

Macro-F1 vs Micro-F1

Macro: compute F1 per class, then average the classes equally. **Micro**: pool all decisions first, then compute one F1. Macro rewards rare classes.

Example. The class "Fact" has 900 sentences with F1 0.90. "Dissent" has 10 with F1 0.30. Macro-F1 is $(0.90 + 0.30)/2 = 0.60$. Micro-F1 stays near 0.89, because "Fact" dominates the pooled counts.

Subset accuracy and Micro-F1@7

Subset accuracy: a multi-label prediction counts only if the whole set is exactly right. **@7**: only the top 7 retrieved items are scored.

Example. Gold $\{302, 120B\}$, predicted $\{302\}$: subset accuracy is 0, though partly right. Retrieval: 5 relevant cases, 3 of them in the top 7, so $P@7 = 3/7 = 0.43$ and $R@7 = 3/5 = 0.60$.

Class imbalance

Class imbalance

Some labels are far more frequent than others. A model can score well on accuracy by always predicting the common class, yet be useless for the rare ones. Fixes include class weights, resampling, and tuned thresholds.

Example. In CJPE, suppose 80% of appeals are dismissed. "Always predict dismissed" gets 80% accuracy. Its Macro-F1 is only about 0.44, because recall on "allowed" is 0. A class weight of 4 on "allowed" makes each of those mistakes cost four times as much.

BERTScore (Phase 1 and Phase 6 primary metric)

1. A pretrained encoder turns every token of the reference r and the hypothesis h into a **contextual embedding**: a vector whose value depends on the surrounding words.
2. Each token is matched to its most similar token on the other side, by **cosine similarity**.
3. These matches give recall, precision and F1:

$$R = \frac{1}{|r|} \sum_i \max_j \cos(\mathbf{r}_i, \mathbf{h}_j)$$

$$P = \frac{1}{|h|} \sum_j \max_i \cos(\mathbf{r}_i, \mathbf{h}_j), \quad F_1 = \frac{2PR}{P+R}$$

Example

Reference: "the petition was rejected".
Hypothesis: "the appeal was dismissed".

Exact-word metrics see only "the" and "was" in common. BERTScore finds that *dismissed* and *rejected*, and *appeal* and *petition*, have similar embeddings. It gives a high score for the right *meaning*.

Cosine similarity

$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$: 1 means the same direction and 0 means unrelated. Example: $\cos([1, 0], [1, 1]) = 1/\sqrt{2} \approx 0.71$.

ROUGE-L and BLEU-4

ROUGE-L (longest common subsequence)

Find the longest sequence of words that appears in both texts in the same order, though not necessarily adjacent. Precision and recall are its length over each text's length.

Example. Ref: "the appeal is dismissed with costs". Hyp: "the appeal is allowed with costs". The LCS is "the appeal is with costs", 5 words. So $P = R = 5/6$ and ROUGE-L $= 0.83$, even though the *meaning* is reversed.

BLEU-4

The geometric mean of 1-, 2-, 3- and 4-gram precisions, times a **brevity penalty** $\mathrm{BP} = e^{1-|r|/|h|}$ when the hypothesis is shorter than the reference.

Example. In the same pair, none of the 3 hypothesis 4-grams ("the appeal is allowed", ...) appears in the reference. Unsmoothed BLEU-4 is therefore 0, though 5 of 6 words match. A hypothesis half the reference length gets $\mathrm{BP} = e^{-1} = 0.37$.

Perplexity vs generation metrics, and bits per word

Why loss is not the whole story

Validation loss measures how well the model predicts the *true* next token. BERTScore measures what the model *actually writes* after a decoding choice. The two can disagree.

Example. Two models have loss 2.40 and 2.45. With temperature 1.2 the better one rambles; with temperature 0.7 the worse one writes tight legal prose. The decoding choice decides the BERTScore ranking.

Bits per word

Total loss over a text, in bits, divided by its number of *words*. Unlike per-token loss, it is comparable across tokenizers.

Example. Tokenizer A: 1.3 tokens/word at 2.0 nats/token, so $1.3 \times 2.0 / \ln 2 = 3.75$ bits/word. Tokenizer B: 1.1 tokens/word at 2.3 nats/token, so 3.65 bits/word. B is better, though its per-token loss looks worse.

Tokens, corpus and budget

Token and corpus

A **token** is the unit a model reads: a word, a piece of a word, or a symbol. A **corpus** is the collection of training text.

Example. "appellant's" may become three tokens: `appell`, `ant` and `'s`. The Phase-1 corpus has more than 4.5 billion tokens across four sources.

Token budget and data mixing

The **budget** is how many tokens you choose to train on. **Mixing ratios** set each source's share of it.

Example. A 200M budget with Acts at 0.10 means 20M tokens of statutes. Setting Acts to 0 is an *ablation*: does the model's judgment writing improve or suffer without them?

File formats and streaming

JSON and JSON Lines

JSON stores nested data as one document. **JSON Lines** (JSONL) stores one complete JSON object per line.

Example. JSON: `[{"text":"A"},{"text":"B"}]`. JSONL has the same data on two lines: `{"text":"A"}` then `{"text":"B"}`. JSONL can be read line by line; a JSON array needs a streaming parser.

Streaming (ijson)

Read a file record by record, keeping only the current record in memory. The whole file is never loaded.

Example. The 10.7 GB High Courts file would not fit in a 12 GB Colab RAM with Python overhead. Streaming reads one judgment, writes it out, and discards it, using a few megabytes.

Splitting without cheating

Data leakage

Validation or test information reaching the model during training. It makes scores look better than they will be on truly new data.

Example. One judgment is cut into 1,024-token blocks. Block 3 goes to training and block 4 to validation. The model has already seen the case facts, so validation loss is too optimistic.

Document-level split

Assign whole documents to train or validation, never both. We use this.

Example. Judgment #881 goes entirely to validation. Its prompts in the dev set are therefore genuinely unseen, just as the hidden test prompts are.

Storage for fast training

Token bin and uint16

All token IDs are written into one flat binary file. `uint16` is an unsigned 16-bit integer (0 to 65,535), so each token takes 2 bytes.

Example. 200M tokens take 400 MB as uint16, and 1.6 GB as Python's default 64-bit integers. A 32k vocabulary fits under 65,535, so uint16 is enough.

Memory map (memmap)

Treat a file on disk as if it were an array in memory. The operating system loads only the pages you touch.

Example. Training asks for tokens 5,000,000 to 5,001,025. Only those 2 KB are read from disk, not the whole 400 MB file.

Special tokens and dev-set construction

End-of-text token

A special token, <|endoftext|>, placed between documents. It tells the model that one text ends here and another begins.

Example. "... appeal dismissed. <|endoftext|> IN THE HIGH COURT OF DELHI ...". The model learns not to carry names from one case into the next. During Phase-1 generation we *ban* it, so the model never stops early.

Dev set

Our own labelled stand-in for the hidden test set. It has the same format, built from held-out documents.

Example. Take held-out judgment #881, choose a random 640-word window, and use the first 512 words as the prompt. The next 128 words become the reference answer.

Tokenizer and vocabulary

Tokenizer

Converts text to a list of integer IDs, and back again. The model only ever sees the integers.

Example. "The appellant" becomes `[412, 2871]` and then decodes back to "The appellant". Every other stage works on those numbers.

Vocabulary and special tokens

The fixed set of tokens the tokenizer knows. Its size V sets the width of the embedding table and the output layer.

Example. With $V = 32{,}000$, token 2871 might be "Ġappellant" (Ġ marks a leading space). A larger vocabulary means shorter sequences but more embedding parameters.

Byte-Pair Encoding (BPE), step by step

Idea: start from single characters (or bytes). Repeatedly merge the *most frequent adjacent pair* into a new token until the vocabulary reaches its target size.

Toy corpus word counts: *appeal* $\times 10$, *appellant* $\times 8$, *apply* $\times 5$.

Merge	Pair (count)	New token
1	a + p (23)	`ap`
2	ap + p (23)	`app`
3	app + e (18)	`appe`
4	appe + l (18)	`appel`
...		`appeal`, `appell`

Why this works

Frequent words become **one token**. Rare words are split into known pieces:

appellant $\rightarrow$ `appell` + `ant`
appellate $\rightarrow$ `appell` + `ate`

The model can reuse what it learned about `appell`. No word is ever "unknown".

Byte-level BPE and the unknown-token problem

Byte-level BPE

BPE that starts from the 256 possible bytes instead of characters. Any text can be encoded, because every string is a sequence of bytes.

Example. A judgment quoting the Devanagari name "*Ram*", or the symbol "§", is split into its UTF-8 bytes and still encoded. Nothing is lost.

Unknown token (unk) and OOV

Word-level tokenizers map words outside their vocabulary (*out-of-vocabulary*) to a single `<unk>` token. Byte-level BPE never needs it.

Example. A word-level tokenizer trained on news would turn "Vakalatnama" into `<unk>`, and the model could never write it. Our legal BPE learns it, or splits it into pieces.

Fertility and GPT-2's tokenizer

Fertility (tokens per word)

The average number of tokens per word. Lower fertility means shorter sequences: more text fits in the context window, and each training step covers more words.

Example. A prompt has 512 words. At fertility 1.35 it is about 690 tokens, which fits in 1,024 with room for the answer. At fertility 1.8 it is 920 tokens, leaving only 104 for the continuation.

GPT-2 tokenizer

A 50,257-token byte-level BPE trained on web text. The starter notebook uses it. It is allowed, because it is a tokenizer and not pretrained model weights.

Example. The web is full of "Ġthe" and "Ġemail". Legal words like "respondent" may cost 2 to 3 GPT-2 tokens but only 1 in a legal BPE. That is exactly what `-compare-gpt2` measures.

What a language model does

Language model (LM)

A model that assigns probabilities to text. Equivalently, it gives a probability for the next token, given the tokens so far.

Example. After "The appeal is hereby", a good legal LM gives "dismissed" 0.45, "allowed" 0.30, "disposed" 0.15, and "banana" nearly 0.

Autoregressive / causal LM

Generates one token at a time. Each new token is appended to the input, and the model predicts again. "Causal" means each position may use only the past.

Example. "The" $\rightarrow$ "appeal" $\rightarrow$ "is" $\rightarrow$ "dismissed": four passes, each feeding on the previous output. This is how the 128-word continuation is written.

Logits and softmax

Logits

The raw, unnormalised scores the model outputs, one per vocabulary token. Higher means more likely. They can be negative.

Example. For the next word after "The appeal is": dismissed 2.0, allowed 1.0, pending 0.0.

Softmax

Turns logits z into probabilities that are positive and sum to 1: $p_i = e^{z_i} / \sum_j e^{z_j}$.

Example. $e^2 = 7.39$, $e^1 = 2.72$, $e^0 = 1$, and the sum is 11.11. So $p = [0.665, 0.245, 0.090]$ for dismissed, allowed and pending.

Cross-entropy loss and perplexity

Cross-entropy loss

The negative log-probability the model gave the *correct* token, averaged over positions: $\mathcal{L} = -\frac{1}{T}\sum_t \log p(x_t \mid x_{<t})$. Lower is better.

Example. If the true word is "dismissed" ($p = 0.665$), the loss is $-\ln 0.665 = 0.41$. If it was "pending" ($p = 0.09$), the loss is $-\ln 0.09 = 2.41$. Confident wrong guesses cost the most.

Perplexity (PPL)

$e^{\mathcal{L}}$: the effective number of equally likely choices the model is "torn between" at each step.

Example. Loss 3.0 gives PPL $e^3 \approx 20$: like choosing among 20 words each time. Loss 2.3 gives about 10. A uniform guess over 32,000 tokens has PPL 32,000.

Context, prefix and continuation loss

Context window (block size)

The maximum number of tokens the model reads at once. Ours is 1,024.

Example. The 690-token prompt plus about 180 generated tokens is 870, which fits in 1,024. A 3,000-token judgment does not fit, and must be truncated or chunked.

Prefix and continuation loss

The **prefix** is the given context; the model is scored only on the tokens after it. With a prefix of p tokens, loss is averaged over positions $p+1, \ldots, T$.

Example. With $p = 768$ in a 1,025-token block, the model reads 768 tokens and is scored on the next 257. This mirrors Phase 1 exactly: read the prompt, then write.

Parameters and what “from scratch” means

Parameters (weights)

The learned numbers inside the model.
Embedding parameters are the token table.
Non-embedding parameters are everything else.

Example. A1 has 82.1M parameters: 24.6M in the $32{,}064 \times 768$ embedding table and 57.5M elsewhere (the blocks plus the position table). The rules require 50M to 300M.

From scratch vs pretrained

From scratch: weights start random, and all knowledge comes from our training. **Pretrained**: weights downloaded from a model someone else trained. That is forbidden here.

Example. Before training, our model writes “appeal xyl the the Qz”. After 200M tokens of legal text it writes “appeal is dismissed with costs”. Every bit of that came from the provided corpus.

Embeddings

Token embedding

A lookup table that maps each token ID to a learned vector of d numbers. Tokens with similar meaning end up with similar vectors.

Example. Token 2871 ("appellant") maps to row 2871 of a $32{,}000 \times 768$ table, a 768-number vector. After training, "appellant" and "petitioner" sit close together.

Tied embeddings

The output layer that scores the next token reuses the input embedding table, instead of learning a separate one.

Example. This saves $32{,}064 \times 768 = 24.6\text{M}$ parameters. It also makes sense: if "appellant" *means* vector v, the model should predict it when its hidden state looks like v.

Self-attention: queries, keys and values

Each token makes three vectors from its embedding:

- **Query** q: what am I looking for?
- **Key** k: what do I contain?
- **Value** v: what do I pass on if chosen?

$$\text{Attention}(Q, K, V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_h}}\Big)V$$

Each output is a weighted average of the values. The weights come from query–key similarity. Dividing by $\sqrt{d_h}$ keeps scores from growing with the vector size.

Example

"The **respondent** filed a reply, but **he** did not appear."

The query of "he" matches the key of "respondent" strongly, with a score of 0.8. So "he" mostly copies the value of "respondent", and the model now "knows" who "he" is.

Numerically: $q \cdot k = 16$ with $d_h = 64$ gives $16/8 = 2.0$ before the softmax.

Multi-head attention and head dimension

Multi-head attention (MHA)

Run several attention operations (*heads*) in parallel, each on a slice of the vector of size $d_h = d/H$, and concatenate their outputs. Different heads learn different relationships.

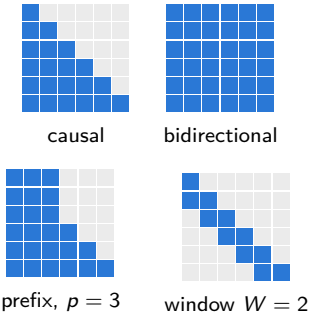
Example. With $d = 768$ and $H = 12$ heads, each head works on $d_h = 64$ numbers. In a judgment, one head might link pronouns to parties ("he" $\rightarrow$ "respondent"). Another might link "Section" to its number ("302"). A third might track the case citation being discussed.

Attention masks

A **mask** decides which keys each query may see. Blocked scores are set to $-\infty$, so their softmax weight is 0.

- **Causal**: see only the past. This is needed for generation, or the model could peek at the answer.
- **Bidirectional**: see everything. Good for *reading* a prompt.
- **Prefix**: bidirectional inside the prompt, causal after it.
- **Sliding window**: see only the last W tokens.

Example. When predicting word 5, a causal mask stops the model from seeing word 5 itself. Without it, training would be trivial copying.



Residual connections and normalisation

Residual connection

Add a layer's input to its output: $x \leftarrow x + f(x)$. The layer only has to learn a *correction*, and gradients flow straight through deep stacks.

Example. Like editing a draft with tracked changes rather than rewriting it from scratch each round. If a layer has nothing useful to add, $f(x) \approx 0$ and the draft passes through unchanged.

LayerNorm and pre-LN

LayerNorm rescales each vector to mean 0 and variance 1, then applies a learned scale and shift. **Pre-LN** normalises *before* each sub-layer, which trains more stably.

Example. $x = [2, 4, 6]$ has mean 4 and standard deviation 1.63, giving $[-1.22, 0, 1.22]$. The numbers keep their shape, but no layer can blow them up to $[2000, 4000, 6000]$.

RMSNorm and the feed-forward network

RMSNorm

Divide by the root-mean-square only; there is no mean subtraction and no bias: $x/\sqrt{\frac{1}{d}\sum x_i^2 + \epsilon} \odot g$. It is cheaper and works as well in practice.

Example. $x = [2, 4, 6]$: $\text{rms} = \sqrt{(4 + 16 + 36)/3} = 4.32$, giving [0.46, 0.93, 1.39]. Compare LayerNorm's $[-1.22, 0, 1.22]$: the shape is similar, with less arithmetic.

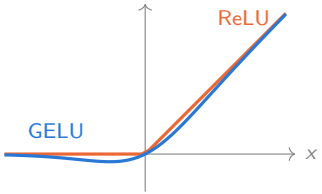
MLP / feed-forward network (FFN)

After attention mixes information *between* tokens, a two-layer network processes each token *on its own*: expand to $4d$, apply a non-linearity, project back to d.

Example. Attention gathers "he = respondent". The MLP then works with that fact, for example "a respondent who did not appear suggests an *ex parte* order". Roughly two thirds of a block's parameters live here.

Activation functions

- **ReLU**: $\max(0, x)$. $\mathrm{ReLU}(-2) = 0$, $\mathrm{ReLU}(3) = 3$. Simple, but "dead" for negatives.
- **GELU**: $x \cdot \Phi(x)$, a smooth ReLU. $\mathrm{GELU}(-0.5) \approx -0.15$, so small negatives leak through.
- **SiLU**: $x \cdot \sigma(x)$. $\mathrm{SiLU}(1) = 0.73$.
- **SwiGLU**: $W_2\big(\mathrm{SiLU}(W_g x) \odot W_u x\big)$. One branch computes content; the other acts as a learned *gate* that says how much of it passes.



SwiGLU as a valve

Content says "this token is a date". The gate says "0.1: dates matter little here". The output is a mostly closed valve.

Dropout and weight initialisation

Dropout

During training, randomly zero a fraction p of activations, so the network cannot rely on any single unit. It is turned off at inference.

Example. With $p = 0.1$, one in ten features is hidden at each step. It is like studying with random lines of your notes covered, so you must understand the case and not memorise one line. With 200M tokens and one pass, overfitting is rare, so we default to 0.

Weight initialisation

The random starting values. We draw from $\mathcal{N}(0, 0.02^2)$ and shrink residual output layers by $1/\sqrt{2L}$ so the sum of L layers does not explode.

Example. With 8 layers, the output projections start with standard deviation $0.02/\sqrt{16} = 0.005$. Adding 16 residual updates then keeps the signal near its starting size.

Encoder, decoder and cross-attention

Encoder vs decoder

An **encoder** reads a whole input with bidirectional attention and produces vectors. A **decoder** generates text causally, one token at a time.

Example. An encoder is the judge reading the full case file, where every page may refer to every other. A decoder is the judge writing the order, word by word, unable to see words not yet written.

Cross-attention

Decoder queries attend to the *encoder's* keys and values. This is how the writer consults the reader's notes.

Example. While writing "the appeal is _____", the decoder's query searches the encoded prompt. It finds that earlier text said "no merit in the submissions", so it writes "dismissed".

The three architecture families

Family	How it works	Legal analogy
Decoder-only (GPT, A1, A4)	one causal stack reads and writes	a clerk who reads the file left to right and keeps writing
Encoder–decoder (T5, BART, A2)	separate reader and writer, joined by cross-attention	a researcher summarises the file; a drafter writes from the notes
Prefix-LM (UniLM, GLM, A3)	one stack; the mask makes it read the prompt bidirectionally, then write causally	one lawyer reads the whole brief first, then drafts

Same sentence, three views

For "the appellant filed an appeal | against the order", with the prompt before "|": A1 reads "appellant" without seeing "appeal". A2 and A3 let "appellant" see "appeal" while reading. All three must write "against the order" causally.

Positional information (1)

Why positions are needed

Attention itself ignores order. Without position information, a sentence is just a bag of tokens.

Example. "The appellant sued the respondent" and "The respondent sued the appellant" contain the same tokens. Only positions tell the model who sued whom.

Learned absolute positions (A1)

A second lookup table with one learned vector per position $0 \ldots 1023$, added to the token embedding.

Example. Position 5 always adds the same vector P_5. The model has no vector for position 1,500, so it *cannot* read past 1,024 tokens. It is a ruler with only 1,024 marks.

Positional information (2)

Sinusoidal positions (A2)

A fixed table of sine and cosine waves at many frequencies, with nothing learned. Each position gets a unique pattern.

Example. Like a clock with many hands. The second hand distinguishes nearby positions, and the hour hand distinguishes distant ones. Together they give every position a unique reading, even beyond the training length.

RoPE (rotary position embedding, A4)

Rotate each query and key by an angle proportional to its position. The dot product then depends only on the *difference* in positions.

Example. Two clock hands turned by $30^\circ \times$position. Tokens 3 and 5, or tokens 103 and 105, differ by the same 60°. So "Section" followed 2 tokens later by "302" looks the same anywhere in the judgment.

Positional information (3): ALiBi and extrapolation

ALiBi (A3)

No position vectors at all. Each head subtracts a penalty proportional to distance from its attention score: $-m_h\,|i-j|$. Heads with small slopes m_h look far; heads with large slopes look nearby.

Example. Head slope $m = 0.5$. A key 3 tokens away loses 1.5 from its score; one 20 tokens away loses 10 and is effectively ignored. A head with $m = 0.01$ loses only 0.2 at distance 20, so it can link distant clauses.

Extrapolation

Working well on inputs longer than those seen in training.

Example. Trained on 1,024 tokens, and asked about 1,500. Learned positions have no row 1,500 and fail. ALiBi only sees distances it knows how to penalise. This matters for long judgments in Phases 4 to 7.

Grouped-query attention (GQA)

Grouped-query attention

Use many query heads but fewer key/value heads; each group of query heads shares one K/V head. Multi-query attention (MQA) is the extreme case of a single K/V head.

Example. A4 has 12 query heads and 4 K/V heads, so 3 queries share each. It is like 12 students sharing 4 textbooks: everyone asks their own question, but only 4 books need to be carried. The K/V projections and the generation cache shrink $3\times$. For 800 cached tokens over 10 layers, keys plus values hold $800 \times 10 \times 2 \times 4 \times 64 = 4.1\text{M}$ numbers instead of 12.3M.

Sliding-window attention and complexity

Sliding-window (local) attention

Each token attends only to the last W tokens. Some *global* layers keep full attention so long-range links survive.

Example. Reading with a ruler that shows only the last 256 tokens. Every third layer lifts the ruler to check the case title and party names at the top.

Big-O cost and FLOPs

$O(T^2)$ means the cost grows with the square of the length. FLOPs (floating-point operations) count raw arithmetic.

Example. Full attention on 1,024 tokens needs about $1{,}024^2 = 1.05$M query–key pairs per head. A window of 256 needs $1{,}024 \times 256 = 0.26$M, a quarter. Doubling to 2,048 tokens quadruples full attention but only doubles windowed attention.

Generation mechanics

KV-cache

Store each layer's keys and values for tokens already processed. Each new token then computes only its own K/V and attends to the cache.

Example. Without a cache, writing word 129 means re-reading all 800 prompt tokens again. That is like re-reading the whole case file before every word of the order. With a cache the judge keeps notes, and each word costs one step.

Prefill and decode

Prefill: run the whole prompt once, in parallel, and fill the cache. **Decode**: generate one token per step using the cache.

Example. Prefill processes about 700 tokens in one pass. Decode then adds 1 token at a time, about 180 steps for 128 words. Prefill is heavy on compute; decode is limited by memory reads.

Padding, rolling cache and padded vocabulary

Padding and left-truncation

In a batch, all sequences must have the same length. **Padding** fills short ones with dummy tokens. We avoid it: we sort prompts by length and cut each batch to its shortest prompt from the left.

Example. A batch has prompts of 702, 705 and 711 tokens. Keeping the last 702 of each drops at most 9 of the oldest tokens. No padding masks are needed.

Rolling cache and padded vocabulary

Rolling cache: local layers keep only the last $W - 1$ keys. **Padded vocabulary**: round V up to a multiple of 64 for faster GPU matrix products.

Example. With $W = 256$, local-layer caches never exceed 255 tokens, however long generation runs. $V = 32{,}000$ is padded to 32,000, and 32,001 would become 32,064. The unused rows are banned during generation.

Gradient descent and the optimizer

Gradient and gradient descent

The gradient is the direction in which the loss increases fastest. Gradient descent moves every weight a small step the *opposite* way: $w \leftarrow w - \eta \nabla \mathcal{L}$.

Example. Walking down a foggy hill by always stepping downhill. The learning rate η is the step length. If the gradient for one weight is $+2$ and $\eta = 0.001$, that weight moves by -0.002.

AdamW

Adam keeps a running average of gradients (**momentum**, $\beta_1 = 0.9$) and of their squares ($\beta_2 = 0.95$), giving each weight its own step size. The "W" is **decoupled weight decay**, applied separately.

Example. Momentum is a ball rolling downhill that does not stop at every small bump. A weight with consistently large gradients takes smaller steps; a rarely updated embedding row, such as for "Vakalatnama", takes larger ones.

Weight decay and regularisation

Weight decay

Each step shrinks weights slightly toward zero: $w \leftarrow w - \eta\lambda w$. Large weights must "earn their keep" by reducing the loss.

Example. With $\lambda = 0.1$ and $\eta = 6 \times 10^{-4}$, each step multiplies a weight by $1 - 0.00006$. A weight of 1.0 that stops being useful fades to about 0.84 after 3,000 steps.

Overfitting, underfitting and regularisation

Overfitting: training loss keeps falling while validation loss rises, because the model memorises. **Underfitting**: both are high, because the model is too small or undertrained. **Regularisation** (dropout, weight decay, early stopping) fights overfitting.

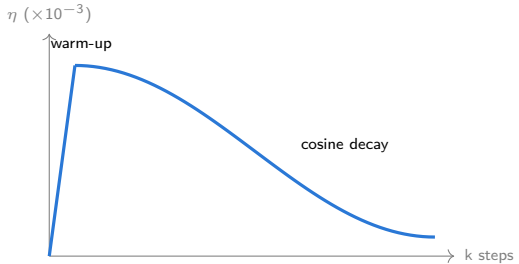
Example. Train loss 1.2 with validation loss 3.5 means memorising judgments verbatim. Train 3.4 with validation 3.5 means the model can learn more, so train longer or make it bigger.

Learning-rate schedule

Warm-up: grow the learning rate linearly from about 0 to its peak over the first steps. Early gradients are noisy, and large steps would wreck the random start.

Cosine decay: then lower it smoothly along a half cosine to a minimum:

$$\eta_t = \eta_{\min} + \tfrac{1}{2}(\eta_{\max} - \eta_{\min})\big(1 + \cos \pi \, \tfrac{t - t_w}{T - t_w}\big)$$



Example

Peak 6×10^{-4}, 200 warm-up steps, 3,000 total. At step 100: 3×10^{-4}. At step 1,600, halfway through the decay: 3.3×10^{-4}. At step 3,000: 6×10^{-5}.

Batches, steps and gradient accumulation

Batch, micro-batch, step and epoch

A **batch** is the examples used for one weight update (a **step**). A **micro-batch** is what fits in GPU memory at once. An **epoch** is one full pass over the data.

Example. 8 blocks of 1,024 tokens form one micro-batch. With 200M training tokens and 65,536 tokens per step, one epoch is about 3,050 steps.

Gradient accumulation

Run several micro-batches, adding up their gradients, before a single update. It gives a big effective batch on a small GPU.

Example. A T4 holds only 8 blocks at once. After 8 micro-batches, the effective batch is $8 \times 8 \times 1{,}024 = 65{,}536$ tokens. It is like filling a large tank with a small bucket: 8 trips, one pour.

Gradient clipping and exploding gradients

Gradient clipping and gradient norm

The **gradient norm** is the overall length of the gradient vector. If it exceeds a limit c, scale the whole gradient down to length c. The direction is kept, but the step is capped.

Example. A rare garbled OCR passage produces a gradient of norm 5.0. With $c = 1.0$, every component is multiplied by $1/5 = 0.2$. One strange batch then cannot throw away hours of training. A log that shows norms constantly far above 1 is a warning sign.

Numerical precision

fp32, fp16 and bf16

Floating-point formats. **fp32**: 32 bits, precise. **fp16**: 16 bits, fast, but the largest value is only 65,504 and tiny values round to zero. **bf16**: 16 bits with the *same range* as fp32, but less precision.

Example. A gradient of 10^{-8} becomes 0 in fp16 (underflow), but survives in bf16. An attention score of 70,000 overflows fp16 to infinity. A T4 GPU supports fp16 but not bf16; an A100 supports both.

Mixed precision and loss scaling (GradScaler)

Do most maths in 16 bits for speed and keep sensitive parts, such as the loss and the weights, in fp32. For fp16, multiply the loss by a large factor before the backward pass so small gradients do not underflow, then divide it back out.

Example. Scale the loss by 1,024: a gradient of 10^{-8} becomes 10^{-5}, which fp16 can hold. If an overflow appears, the scaler skips that step and halves the factor.

Checkpoints, resume and early stopping

Checkpoint (best vs last) and resume

A saved snapshot of the weights, plus optimizer state for resuming. **best.pt** has the lowest validation loss so far. **last.pt** is the most recent. **Resume** continues from last.pt after an interruption.

Example. Colab disconnects at step 2,140. Rerunning the same command loads last.pt, saved at step 2,000, and loses only 140 steps. It is written *atomically*, to a temporary file that is then renamed, so a crash mid-save never corrupts it.

Early stopping and patience

Stop training when validation loss has not improved for a set number of evaluations (the **patience**).

Example. Validation losses are 2.81, 2.79, 2.80, 2.82, 2.83. With patience 3, training stops after the last one and keeps the 2.79 checkpoint.

Throughput, memory and reproducibility

Throughput (tokens/s) and peak memory

Throughput is tokens processed per second. **Peak GPU memory** is the most memory used at any moment. Together they decide what fits in a free Colab session.

Example. At 40,000 tokens/s, 200M tokens take $200\text{M}/40\text{k} = 5{,}000$ s, about 1.4 hours. At 15,000 tokens/s it is 3.7 hours, which may need two sessions and a resume.

Random seed and torch.compile

A **seed** fixes the random number generators, so a run can be repeated exactly. **torch.compile** fuses operations into faster GPU kernels.

Example. Seed 1337 makes step 500 always draw the same blocks, even after a resume. That lets two learning rates be compared on identical data. torch.compile might raise throughput by 20–40% on recent GPUs.

Compute environment

GPU, T4 and CUDA

A **GPU** runs thousands of arithmetic operations in parallel. **CUDA** is NVIDIA's platform for programming it. The **T4** (16 GB) is the free Colab and Kaggle GPU.

Example. Multiplying two 768×768 matrices takes about 0.9 billion operations. A CPU does it in milliseconds, but a GPU runs thousands at once. MPS is Apple's equivalent on a Mac, used for our smoke test.

Colab and Kaggle sessions

Free hosted notebooks with a GPU for a limited time. Sessions can end without warning.

Example. Save runs to Google Drive by setting `paths.runs_dir`. Otherwise a disconnect deletes the local disk, with every checkpoint on it.

Greedy decoding and sampling

Greedy decoding

Always pick the single most likely token ($\tau = 0$). It is deterministic, but often repetitive.

Example. With $p =$ dismissed 0.45, allowed 0.30, disposed 0.15, greedy always writes "dismissed". It can loop: "the court held that the court held that the court held...".

Sampling

Draw the next token at random from the probability distribution. Varied, and more human-like, but it can wander.

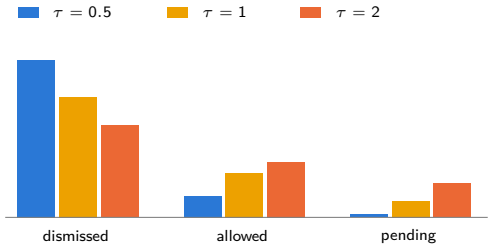
Example. It writes "dismissed" 45% of the time, "allowed" 30% and "disposed" 15%. Occasionally it picks something odd from the long tail. That is why top-k and top-p exist.

Temperature

Temperature τ

Divide the logits by τ before the softmax. $\tau < 1$ sharpens the distribution (safer), $\tau > 1$ flattens it (more creative), and $\tau \to 0$ becomes greedy.

Example. Logits $[2, 1, 0]$ with $\tau = 1$ give $[0.665, 0.245, 0.090]$. With $\tau = 0.5$, they become $[4, 2, 0]$ and $p = [0.867, 0.117, 0.016]$. With $\tau = 2$, they become $[1, 0.5, 0]$ and $p = [0.506, 0.307, 0.186]$.



Lower temperature puts more of the probability on the top choice.

Top-k and top-p (nucleus) sampling

Top-k

Keep only the k most likely tokens, renormalise, and sample among them. The long tail of nonsense is cut off.

Example. Probabilities $[0.50, 0.20, 0.15, 0.10, 0.05]$ with $k = 2$ keep the first two. Renormalised, that is $[0.71, 0.29]$. The rule is fixed: it keeps 2 tokens whether the model is confident or unsure.

Top-p (nucleus)

Keep the smallest set of top tokens whose total probability reaches p. The set size adapts to the model's confidence.

Example. Same probabilities with $p = 0.9$: $0.50 + 0.20 + 0.15 = 0.85 < 0.9$, so add the next token, giving 0.95. Four tokens are kept. For a confident step like $[0.93, 0.04, \ldots]$, only one token is kept.

Repetition control and length control

Repetition penalty and window

Make tokens used recently less likely: divide positive logits by r, multiply negative ones by r. The **window** is how many recent tokens count.

Example. With $r = 1.1$, "court" was just used and has logit 2.0, which becomes 1.82; a logit of -1.0 becomes -1.1. Too high a penalty stops the model repeating legitimate words such as "appellant", which legal text needs.

Degeneration, EOS banning and the 128-word stop

Degeneration is dull, looping or incoherent output. We **ban end-of-text** so the model never stops early, and stop once 129 complete words exist, keeping 128.

Example. If generation stopped at exactly 128 words, the last one could be half a token, like "respond". Waiting for the 129th guarantees word 128 is whole: "respondent".

Beam search (for later phases)

Beam search

Keep the B best partial sequences at each step, extend each, and keep the B best overall. It finds high-probability text, but can be bland. It is not used in our Phase-1 code, but is common for summarisation in Phase 6.

Example. With $B = 2$, after "The appeal" the model keeps "is" (0.6) and "stands" (0.2). It expands both, and keeps the top two complete paths: "is dismissed" ($0.6 \times 0.5 = 0.30$) and "is allowed" ($0.6 \times 0.3 = 0.18$). The path through "stands" is dropped.

Teacher, student and soft targets

Teacher and student

A large trained **teacher** guides the training of a smaller **student**, so the student learns faster and better than from the data alone.

Example. The A1 teacher (82M parameters) trains an A4 student (47M). It is like a senior advocate coaching a junior, explaining how confident to be, not just what the right answer is.

Soft targets and "dark knowledge"

The teacher's full probability distribution, not just the one correct token. It shows which wrong answers are nearly right.

Example. The true next word is "dismissed". The teacher says dismissed 0.6, rejected 0.3, allowed 0.1. The student learns that "rejected" is a near-synonym, information a one-hot label (1, 0, 0) never gives.

The distillation loss

$$\mathcal{L} = \alpha\,\mathcal{L}_{\text{CE}} + (1-\alpha)\,\tau^2\,\mathrm{KL}\big(p_\tau^{T} \,\|\, p_\tau^{S}\big)$$

- **KL divergence**: $\sum_i p_i \ln \frac{p_i}{q_i}$ measures how different two distributions are. It is 0 when they are identical.
- **Distillation temperature** τ: softens both distributions, so small probabilities matter more.
- The τ^2 factor keeps gradients comparable in size.
- α balances real labels against teacher guidance. We use 0.5.

Worked KL example

Teacher $p = [0.7, 0.2, 0.1]$ and student $q = [0.4, 0.4, 0.2]$:

$$0.7 \ln \tfrac{0.7}{0.4} + 0.2 \ln \tfrac{0.2}{0.4} + 0.1 \ln \tfrac{0.1}{0.2}$$

$$= 0.392 - 0.139 - 0.069 = 0.184$$

Training pushes this toward 0: the student learns to put about 70% on the teacher's favourite.

Model compression

Model compression

Making a model smaller or faster while keeping most of its quality. Options include distillation, pruning (removing weights), quantisation (fewer bits per weight), and efficient architectures.

Example. The teacher has 82M parameters and a validation loss of 2.60. A distilled 47M student reaches 2.72; the same student trained without a teacher reaches 2.80. The student keeps most of the quality at 57% of the size, and generates faster. (These numbers are illustrative, not measured.)

Designing experiments

Controlled experiment and ablation

Change **one** factor, hold everything else fixed, and measure the effect. An **ablation** removes one component to see how much it mattered.

Example. Train A1 twice, identical except that the second run drops the Acts source. If validation loss on judgments barely moves but loss on statutes rises, Acts mainly help statute-style text.

Baseline and hyperparameters

A **baseline** is the simple reference system every idea is compared against. **Hyperparameters** are settings chosen by you, not learned, such as learning rate, layers or temperature.

Example. A1 is our baseline. "A4 is better" only means something as "A4 reaches 0.05 lower loss than A1 on the same data and budget". Note: 12 heads versus 16 heads is a hyperparameter change, not a new architecture.

Configuration system

YAML config and inheritance

YAML is a readable text format for settings. **Inheritance** (`inherit: base.yaml`) lets one file extend another and override only what differs.

Example. `a4_efficient.yaml` is only 14 lines. It inherits all the data, tokenizer and training settings from `base.yaml` and adds its own model shape. Changing the token budget in base.yaml changes it for all four models at once.

Command-line overrides

`-set key.sub=value` changes one setting for one run, without editing any file.

Example. `-set train.lr=3e-4 run_name=A1_lr3e-4` makes a new run folder whose saved `config.yaml` records exactly what changed. That record is evidence for the report.

Testing

Smoke test

A quick end-to-end run on tiny data, to check the whole pipeline works before spending hours of GPU time.

Example. `run_smoke_test.sh` builds a fake legal corpus and trains all four models for 40 steps. It then generates, scores and zips the predictions, in a few minutes on a laptop. A broken zip layout is caught here, not on Codabench.

Unit tests: causality and cache correctness

Small automatic checks of one property each. **Causality**: changing a future token must not change past predictions. **Cache correctness**: cached generation must equal a full recomputation.

Example. If a mask bug let position 10 see position 40, the causality test fails at once. Without the test, the model would "cheat" in training, then fall apart at generation time.

Alphabetical index (A–C)

- Ablation – Experiments
- Activation functions – Blocks
- AdaLoRA – LoRA family
- AdamW – Training
- Adapters (Houlsby, Pfeiffer) – Project
- ALiBi – Architectures
- Attention mask – Blocks
- Autoregressive LM – LM basics
- Baseline – Experiments
- Batch, micro-batch – Training
- Beam search – Decoding
- BERTScore – Metrics
- bf16, fp16, fp32 – Training
- Big-O, FLOPs – Architectures
- BitFit – Project
- Bits per word – Metrics
- BLEU-4, brevity penalty – Metrics
- BPE, byte-level BPE – Tokenization
- Budget schedule – LoRA family
- Calibration – Project
- Causal mask – Blocks
- Checkpoint (best, last) – Training
- Chunking – Project
- Class imbalance – Metrics
- Codabench – Project
- Contextual embedding – Metrics
- Context window – LM basics
- Continuation loss – LM basics
- Contrastive learning – Project
- Corpus – Data
- Cosine decay – Training
- Cosine similarity – Metrics
- Cross-attention – Architectures
- Cross-entropy – LM basics
- CUDA, GPU, T4 – Training

Alphabetical index (D–H)

- Data leakage – Data
- Data mixing – Data
- Decoder, decoder-only – Architectures
- Degeneration – Decoding
- Dev set – Data
- Distillation – Distillation
- Document-level split – Data
- Double quantisation – LoRA family
- Dropout – Blocks
- Early stopping – Training
- Embedding (token, tied) – Blocks
- Encoder, encoder–decoder – Architectures
- End-of-text token – Data
- Epoch – Training
- Extractive, abstractive – Project
- Extrapolation – Architectures
- Fertility – Tokenization
- Fine-tuning – Project
- Foundation model – Project
- From scratch – LM basics
- Gated dataset – Project
- GELU, ReLU, SiLU – Blocks
- GPT-2 tokenizer – Tokenization
- Gradient accumulation – Training
- Gradient clipping, norm – Training
- Gradient descent – Training
- GradScaler, loss scaling – Training
- Greedy decoding – Decoding
- Grouped-query attention – Architectures
- Hard negative – Project
- Hierarchical model – Project
- Hyperparameter – Experiments

Alphabetical index (I–M)

- IA^3 – Project
- Importance score (sensitivity) – LoRA family
- Initialisation – Blocks
- Intrinsic rank – LoRA family
- JSON, JSON Lines – Data
- KL divergence – Distillation
- KV-cache – Architectures
- Language model – LM basics
- Layer freezing – Project
- LayerNorm, pre-LN – Blocks
- Learned positions – Architectures
- Learning rate, warm-up – Training
- Left-truncation – Architectures
- Logits – LM basics
- LoRA – LoRA family
- LORMA – LoRA family
- Macro-F1, Micro-F1 – Metrics
- Memory map – Data
- Merging adapters – LoRA family
- Micro-F1@7 – Metrics
- Mixed precision – Training
- Model compression – Distillation
- Momentum – Training
- Multi-head attention – Blocks
- Multi-label – Project
- Multi-query attention – Architectures

Alphabetical index (N–R)

- NER, strict span match – Project
- NF4 (NormalFloat) – LoRA family
- Orthogonality regulariser – LoRA family
- Overfitting, underfitting – Training
- Padded vocabulary – Architectures
- Padding – Architectures
- Paged optimizer – LoRA family
- Parameters – LM basics
- PEFT – Project
- Perplexity – LM basics
- Pooling – Project
- Precision, recall, F1 – Metrics
- Prediction.json – Project
- Prefill, decode – Architectures
- Prefix-LM – Architectures
- Prefix – LM basics
- Prefix tuning – Project
- Pre-training – Project
- Prior case retrieval – Project
- Prompt tuning – Project
- QLoRA – LoRA family
- Query, key, value – Blocks
- Rank inflation – LoRA family
- Rank, low-rank matrix – LoRA family
- Regularisation – Training
- Repetition penalty – Decoding
- Residual connection – Blocks
- Resume – Training
- Rhetorical roles – Project
- RMSNorm – Blocks
- Rolling cache – Architectures
- RoPE – Architectures
- ROUGE-L, LCS – Metrics
- rsLoRA – LoRA family

Alphabetical index (S–Z)

- Sampling – Decoding
- Scaling factor α/r – LoRA family
- Seed – Training
- Self-attention – Blocks
- Self-hosted model – Project
- Sinusoidal positions – Architectures
- Sliding window – Architectures
- Smoke test – Experiments
- Softmax – LM basics
- Soft targets – Distillation
- Special tokens – Tokenization
- Streaming – Data
- Subset accuracy – Metrics
- SVD – LoRA family
- SwiGLU – Blocks
- Teacher, student – Distillation
- Temperature – Decoding
- Threshold – Project
- Throughput – Training
- Token bin, uint16 – Data
- Tokenizer – Tokenization
- Token, token budget – Data
- Top-k, top-p – Decoding
- torch.compile – Training
- Train, val, dev, test – Project
- Unit test – Experiments
- Unknown token, OOV – Tokenization
- Vocabulary – Tokenization
- Weight decay – Training
- YAML, inheritance – Experiments